```
"""Death animation — Option B (Standard).

Fires when player HP hits 0. Plays a 4-second cinematic sequence before
handing off to the existing menu.draw_death_screen():

  1. Slow-motion     — game logic updates at 0.3x for 0.8s, eases back to 1.0x
  2. Camera zoom/pan — y-offset 0 → +25, slow tilt 0° → 12° clockwise
  3. Player ragdoll  — sprite rotates 0° → 85°, weapon ejects 30px, body slides 8px
  4. Blood pool      — continuous blood (8 particles/frame for 0.6s) + growing pool decal (max r=40)
  5. Trinitas flash  — reuses minimap._draw_trinitas_overlay via STATE.death_trinitas_timer
  6. Radio radio cut — audio.fadeout + 0.5s dead air before death screen text appears

Integration points (called from minimap.py and minimap.py):
  trigger_death()           — call when HP <= 0 (replaces the old _burst-only trigger)
  update_death_anim()       — call once per frame BEFORE the main game update; returns the
                              number of game-logic ticks to actually run (slow-mo)
  should_skip_player_draw() — call in draw to decide whether to draw the standing player
  draw_death_overlay()      — call AFTER the main scene draw to apply tilt + fallen body +
                              blood pool + weapon eject + Trinitas overlay

All state lives on STATE so reset() in state.py clears it cleanly.
"""
import math
import random

import pygame

from config import WIDTH, HEIGHT, C_BLOOD
from state import STATE
import effects
from config import audio

# ── Timing (all in frames @ 60fps) ─────────────────────────
DEATH_ANIM_DURATION = 240      # 4.0s total — matches legacy death_timer
SLOWMO_PHASE_END = 48          # 0.8s — slow-mo holds at 0.3x within here
SLOWMO_EASE_END = 96           # 1.6s — by here, time_scale is back to 1.0
SLOWMO_MIN = 0.25              # slow-mo floor
CAMERA_ZOOMPAN_FRAMES = 60     # 1.0s — y-offset and tilt reach target
CAMERA_Y_TARGET = 25.0
CAMERA_TILT_TARGET = 12.0      # degrees, clockwise
RAGDOLL_FRAMES = 30            # 0.5s — player rotates 0° → 85°
RAGDOLL_ROTATION_TARGET = 85.0
RAGDOLL_SLIDE_DISTANCE = 8.0   # body slides this far in fall direction
BLOOD_DURATION = 36            # 0.6s of continuous blood particle spawn
BLOOD_PARTICLES_PER_FRAME = 8
BLOOD_POOL_MAX_RADIUS = 40.0
BLOOD_POOL_GROW_FRAMES = 60    # 1.0s — pool grows from 0 to max
WEAPON_EJECT_DELAY = 6         # weapon clatters 0.1s after death (so it reads as a consequence)
WEAPON_EJECT_DISTANCE = 30.0
TRINITAS_RISE_FRAMES = 60      # 1.0s rise
TRINITAS_HOLD_FRAMES = 30      # 0.5s hold
TRINITAS_FADE_FRAMES = 30      # 0.5s fade
TRINITAS_TOTAL = TRINITAS_RISE_FRAMES + TRINITAS_HOLD_FRAMES + TRINITAS_FADE_FRAMES  # 120
RADIO_CUT_FADE_MS = 200        # quick fadeout of any playing radio/voice


def _ease_out_cubic(t):
    """Decelerating ease — feels weighty for a fall."""
    t = max(0.0, min(1.0, t))
    return 1.0 - (1.0 - t) ** 3


def _ease_in_out_cubic(t):
    """Smooth slow-in slow-out for camera/time-scale."""
    t = max(0.0, min(1.0, t))
    return 4 * t * t * t if t < 0.5 else 1 - (-2 * t + 2) ** 3 / 2


def trigger_death():
    """Initialises the death animation. Safe to call multiple times — only fires once.

    Replaces the old pattern of:
        STATE.player_dead = True
        STATE.death_timer = 240
        effects.shake(20)
        _burst(cx, cy, C_BLOOD, 40, speed_mul=3.0)
    """
    if STATE.death_anim_phase != 0:
        return  # already triggered

    STATE.player_dead = True
    STATE.death_timer = DEATH_ANIM_DURATION
    STATE.death_anim_phase = 1
    STATE.death_anim_timer = 0
    STATE._death_frame_accumulator = 0.0

    # Time scale starts at the slow-mo floor.
    STATE.death_time_scale = SLOWMO_MIN

    # Snapshot death position (world coords) for blood pool + weapon eject + body slide.
    if STATE.player is not None:
        cx = STATE.player.rect.centerx
        cy = STATE.player.rect.centery
        STATE.death_blood_pool_pos = (cx, cy)
        STATE.death_blood_pool_radius = 0.0
        STATE.death_blood_bleed_timer = BLOOD_DURATION

        # Weapon ejects 30px in the player's facing direction.
        # Rotation: weapon lies flat, rotated 90° from its in-hand orientation.
        facing = getattr(STATE.player, "angle", 0.0)
        STATE.death_weapon_ejected = False  # set True after WEAPON_EJECT_DELAY
        eject_dx = math.cos(facing) * WEAPON_EJECT_DISTANCE
        eject_dy = math.sin(facing) * WEAPON_EJECT_DISTANCE
        STATE.death_weapon_pos = (cx + eject_dx, cy + eject_dy)
        STATE.death_weapon_rotation = math.degrees(facing) + 90.0
        STATE.death_weapon_type = getattr(STATE, "current_weapon", "pistol")

        # Body slides 8px in the facing direction (momentum falls forward).
        STATE.death_player_slide_x = math.cos(facing) * RAGDOLL_SLIDE_DISTANCE
        STATE.death_player_slide_y = math.sin(facing) * RAGDOLL_SLIDE_DISTANCE
    else:
        STATE.death_blood_pool_pos = None
        STATE.death_weapon_pos = None

    # Initial kick — one burst (matches legacy feel) + heavy shake.
    if STATE.player is not None:
        cx = STATE.player.rect.centerx
        cy = STATE.player.rect.centery
        effects.burst(cx, cy, C_BLOOD, 40, speed_mul=3.0)
    effects.shake(22)

    # Trinitas flash — reuse minimap's _draw_trinitas_overlay by writing its timer.
    # The overlay reads STATE.minimap_trinitas_timer, so we mirror our value there.
    STATE.death_trinitas_timer = TRINITAS_TOTAL
    STATE.minimap_trinitas_timer = TRINITAS_TOTAL

    # Schedule Trinitas audio (rises with the visual).
    try:
        audio.schedule(0, "trinitas", volume=0.8)
    except Exception:
        pass

    # Radio radio cut — fade out any playing radio/voice channel quickly.
    try:
        audio.fadeout("radio", RADIO_CUT_FADE_MS)
        audio.fadeout("voice", RADIO_CUT_FADE_MS)
    except Exception:
        # Some audio backends don't have these channels — silently ignore.
        pass


def update_death_anim():
    """Advance all death-anim state by one frame.

    Returns the number of game-logic ticks the caller should run this frame
    (0 or 1). At full speed this is 1; during slow-mo it's often 0 (so only
    every Nth tick is run) via the accumulator.

    Call this at the TOP of game_update(), before any other logic. The
    caller should wrap the actual entity updates in 'for _ in range(ticks):'
    so bullets/enemies/etc. advance at the slowed rate while visual effects
    (particles, camera) advance at full rate.
    """
    if STATE.death_anim_phase != 1:
        return 1  # not dying — run normally

    STATE.death_anim_timer += 1
    t = STATE.death_anim_timer  # frames since trigger

    # ── 1. Slow-motion time scale ─────────────────────────
    # 0 → SLOWMO_PHASE_END: hold at SLOWMO_MIN (0.25x)
    # SLOWMO_PHASE_END → SLOWMO_EASE_END: ease from SLOWMO_MIN back to 1.0
    # After SLOWMO_EASE_END: full speed.
    if t <= SLOWMO_PHASE_END:
        STATE.death_time_scale = SLOWMO_MIN
    elif t <= SLOWMO_EASE_END:
        ease_t = (t - SLOWMO_PHASE_END) / (SLOWMO_EASE_END - SLOWMO_PHASE_END)
        STATE.death_time_scale = SLOWMO_MIN + (1.0 - SLOWMO_MIN) * _ease_in_out_cubic(ease_t)
    else:
        STATE.death_time_scale = 1.0

    # Accumulator pattern: at 0.3x we run a logic tick every 3-4 frames.
    STATE._death_frame_accumulator += STATE.death_time_scale
    ticks = 0
    while STATE._death_frame_accumulator >= 1.0:
        STATE._death_frame_accumulator -= 1.0
        ticks += 1
    # Cap at 1 tick per frame — we never want to run faster than realtime.
    ticks = min(ticks, 1)

    # ── 2. Camera zoom/pan (y-offset + tilt) ─────────────────
    if t <= CAMERA_ZOOMPAN_FRAMES:
        ease_t = t / CAMERA_ZOOMPAN_FRAMES
        e = _ease_out_cubic(ease_t)
        STATE.death_camera_y_offset = CAMERA_Y_TARGET * e
        STATE.death_camera_tilt = CAMERA_TILT_TARGET * e
    else:
        STATE.death_camera_y_offset = CAMERA_Y_TARGET
        STATE.death_camera_tilt = CAMERA_TILT_TARGET

    # ── 3. Player ragdoll (rotation + slide) ─────────────────
    if t <= RAGDOLL_FRAMES:
        ease_t = t / RAGDOLL_FRAMES
        e = _ease_out_cubic(ease_t)
        STATE.death_player_rotation = RAGDOLL_ROTATION_TARGET * e
    else:
        STATE.death_player_rotation = RAGDOLL_ROTATION_TARGET
    # Slide is applied in draw_death_overlay (uses the slide offset
    # baked in at trigger time) and applies it scaled by the
    # rotation progress so the body settles as it falls.
    slide_progress = min(1.0, t / RAGDOLL_FRAMES)
    # Already stored at trigger time; nothing to update here.

    # ── 4a. Weapon eject ────────────────────────────────
    if t >= WEAPON_EJECT_DELAY and not STATE.death_weapon_ejected:
        STATE.death_weapon_ejected = True
        # Play a metal clatter sound if available.
        try:
            audio.play("sfx_clatter", volume=0.6)
        except Exception:
            pass

    # ── 4b. Blood pool + continuous blood ───────────────────
    if STATE.death_blood_pool_pos is not None:
        if t <= BLOOD_POOL_GROW_FRAMES:
            ease_t = t / BLOOD_POOL_GROW_FRAMES
            STATE.death_blood_pool_radius = BLOOD_POOL_MAX_RADIUS * _ease_out_cubic(ease_t)
        else:
            STATE.death_blood_pool_radius = BLOOD_POOL_MAX_RADIUS

        if STATE.death_blood_bleed_timer > 0:
            STATE.death_blood_bleed_timer -= 1
            px, py = STATE.death_blood_pool_pos
            # 8 small blood particles per frame for 0.6s — reads as continuous bleed.
            for _ in range(BLOOD_PARTICLES_PER_FRAME):
                effects.burst(
                    px + random.randint(-4, 4),
                    py + random.randint(-4, 4),
                    C_BLOOD, count=1, speed_mul=0.5,
                    life_mul=0.8, life_mul_y=0.8,
                )

    # ── 5. Trinitas timer (drives _draw_trinitas_overlay) ─────────
    if STATE.death_trinitas_timer > 0:
        STATE.death_trinitas_timer -= 1
        # Mirror into minimap_trinitas_timer so the existing overlay draws.
        STATE.minimap_trinitas_timer = max(
            getattr(STATE, "minimap_trinitas_timer", 0),
            STATE.death_trinitas_timer,
        )

    # ── 6. Death timer countdown → triggers death screen ────────
    # Legacy behaviour: when death_timer hits 0, set request_death_screen.
    if STATE.death_timer > 0:
        STATE.death_timer -= 1
        if STATE.death_timer == 0:
            if not STATE.minimap_end_time:
                STATE.minimap_end_time = pygame.time.get_ticks()
            STATE.request_death_screen = True
            STATE.death_anim_phase = 2  # done — screen takes over

    return ticks


def should_skip_player_draw():
    """Returns True if the normal standing-player draw should be skipped.

    During death we draw a fallen-body silhouette in draw_death_overlay()
    instead, so the standing player sprite doesn't show through the rotation.
    """
    return STATE.death_anim_phase >= 1


def draw_death_overlay(surface, fxo, fyo):
    """Draw all death-anim visual layers on top of the rendered scene.

    Called AFTER the main scene draw (so the world is already on `surface`)
    but BEFORE the HUD. Applies:
      - camera tilt (rotates the entire screen contents)
      - camera y-offset (already applied by caller via STATE.death_camera_y_offset)
      - blood pool decal under the body
      - ejected weapon
      - fallen body silhouette (rotated player)
      - Trinitas overlay (delegated to minimap._draw_trinitas_overlay)

    `fxo`, `fyo` are the current camera/fx offset in world coords.
    """
    if STATE.death_anim_phase == 0:
        return

    # ── Blood pool decal (drawn first so body sits on top) ──────────
    if STATE.death_blood_pool_pos is not None and STATE.death_blood_pool_radius > 0:
        wx, wy = STATE.death_blood_pool_pos
        sx = int(wx - fxo)
        sy = int(wy - fyo)
        r = int(STATE.death_blood_pool_radius)
        if r > 0:
            # Soft dark-red ellipse, slightly wider than tall (top-down perspective).
            pool = pygame.Surface((r * 2 + 4, r * 2 + 4), pygame.SRCALPHA)
            pygame.draw.ellipse(pool, (90, 8, 8, 200), (0, 0, r * 2 + 4, r * 2 + 4))
            # Darker inner pool.
            inner_r = max(1, r - 6)
            pygame.draw.ellipse(pool, (60, 4, 4, 230),
                                (6, 6, inner_r * 2 - 4, inner_r * 2 - 4))
            surface.blit(pool, (sx - r - 2, sy - r - 2))

    # ── Ejected weapon (rotated on the ground) ───────────────
    if STATE.death_weapon_ejected and STATE.death_weapon_pos is not None:
        wx, wy = STATE.death_weapon_pos
        sx = int(wx - fxo)
        sy = int(wy - fyo)
        _draw_dropped_weapon(surface, sx, sy,
                             STATE.death_weapon_rotation,
                             STATE.death_weapon_type)

    # ── Fallen body silhouette ─────────────────────────
    if STATE.player is not None and STATE.death_blood_pool_pos is not None:
        _draw_fallen_body(surface, fxo, fyo)

    # ── Trinitas overlay (reuses minimap's drawing) ───────────
    # We mirror STATE.death_trinitas_timer into minimap_trinitas_timer
    # in update_death_anim(), so just call the existing overlay.
    try:
        import minimap as mm
        if hasattr(mm, "_draw_trinitas_overlay"):
            mm._draw_trinitas_overlay(surface)
    except Exception:
        pass


def apply_camera_tilt(surface):
    """Rotate the final screen contents by STATE.death_camera_tilt degrees.

    Call this at the very END of the frame's draw sequence, after everything
    else is rendered. Captures the screen, rotates, blits back. More
    expensive but only runs during the 4s death tick.
    """
    tilt = STATE.death_camera_tilt
    if abs(tilt) < 0.1:
        return  # tilt so small there's no point rotating

    # Capture current screen contents.
    snap = surface.copy()
    # Rotate clockwise (negative angle in pygame's convention).
    rotated = pygame.transform.rotate(snap, -tilt)
    # Fill screen to black so the rotation's empty corners don't show garbage.
    surface.fill((0, 0, 0))
    # Center the rotated surface.
    rect = rotated.get_rect(center=(WIDTH // 2, HEIGHT // 2))
    surface.blit(rotated, rect)


def _draw_dropped_weapon(surface, sx, sy, rotation_deg, weapon_type):
    """Draw a simple top-down silhouette of the dropped weapon."""
    # Render to a small offscreen surface so we can rotate cleanly.
    if weapon_type == "rifle":
        w, h = 30, 6
        col = (50, 50, 55)
        detail = (90, 80, 60)
    else:
        w, h = 14, 6
        col = (60, 60, 65)
        detail = (30, 30, 35)

    weapon_surf = pygame.Surface((w + 4, h + 4), pygame.SRCALPHA)
    # Receiver body.
    pygame.draw.rect(weapon_surf, col, (2, 2, w, h), border_radius=2)
    # Top cover / slide highlight.
    pygame.draw.rect(weapon_surf, detail, (2, 2, w - 2, h // 2), border_radius=1)
    # Grip outline.
    pygame.draw.rect(weapon_surf, (20, 20, 22), (2, 2, w, h), 1, border_radius=2)
    # Magazine (small nub on the underside).
    pygame.draw.rect(weapon_surf, (40, 40, 45), (w // 2 - 2, h, 4, 3))

    rotated = pygame.transform.rotate(weapon_surf, -rotation_deg)
    rect = rotated.get_rect(center=(sx, sy))
    # Soft shadow under the weapon.
    shadow = pygame.Surface((rect.w + 4, rect.h + 4), pygame.SRCALPHA)
    pygame.draw.ellipse(shadow, (0, 0, 0, 60), (0, 0, rect.w + 4, rect.h + 4))
    surface.blit(shadow, (rect.x - 2, rect.y + 1))
    surface.blit(rotated, rect)


def _draw_fallen_body(surface, fxo, fyo):
    """Draw Brinkman's body as a fallen silhouette rotated by death_player_rotation.

    Renders a simple top-down body (torso + head + arms/legs) to an
    offscreen surface, rotates it by the current death rotation, and blits
    it at the death position with the slide offset applied.

    The body color palette matches the player's in-game appearance
    (C_PLAYER_UNIFORM / C_PLAYER_SKIN) so it reads as the same character.
    """
    from config import C_PLAYER_UNIFORM, C_PLAYER_SKIN  # late import — colors may not exist in website mode

    if STATE.death_blood_pool_pos is None or STATE.player is None:
        return

    wx, wy = STATE.death_blood_pool_pos

    # Body slide — body settles 8px in the facing direction as it falls.
    slide_progress = min(1.0, STATE.death_anim_timer / RAGDOLL_FRAMES)
    slide_x = STATE.death_player_slide_x * slide_progress
    slide_y = STATE.death_player_slide_y * slide_progress

    # Render the body to an offscreen surface. Body is drawn pointing "up"
    # (negative y) — head at top, feet at bottom; then we rotate by
    # death_player_rotation. The body faces forward (toward the player's
    # facing direction at death time).
    BODY_W, BODY_H = 20, 34
    pad = 8
    body_surf = pygame.Surface((BODY_W + pad * 2, BODY_H + pad * 2), pygame.SRCALPHA)
    cx_local = (BODY_W + pad * 2) // 2
    cy_local = (BODY_H + pad * 2) // 2

    body_col = C_PLAYER_UNIFORM
    skin_col = C_PLAYER_SKIN
    outline_col = (20, 20, 20)

    # Splayed arms (both arms out at ~45° from the torso — ragdoll feel pose).
    arm_col = body_col
    # Left arm
    pygame.draw.line(body_surf, arm_col,
                     (cx_local - 4, cy_local - 6),
                     (cx_local - 12, cy_local + 4), 3)
    # Right arm
    pygame.draw.line(body_surf, arm_col,
                     (cx_local + 4, cy_local - 6),
                     (cx_local + 12, cy_local + 4), 3)
    # Hands (small skin circles at the ends of the arms).
    pygame.draw.circle(body_surf, skin_col, (cx_local - 12, cy_local + 4), 2)
    pygame.draw.circle(body_surf, skin_col, (cx_local + 12, cy_local + 4), 2)

    # Torso — rounded rectangle, slightly tapered toward the waist.
    pygame.draw.rect(body_surf, body_col,
                     (cx_local - 7, cy_local - 10, 14, 16),
                     border_radius=4)
    # Vest details (slightly darker strip down the centre).
    vest_col = tuple(max(0, c - 20) for c in body_col)
    pygame.draw.rect(body_surf, vest_col,
                     (cx_local - 2, cy_local - 8, 4, 12),
                     border_radius=2)

    # Legs — splayed slightly, feet apart.
    leg_col = tuple(max(0, c - 30) for c in body_col)
    # Left leg
    pygame.draw.rect(body_surf, leg_col,
                     (cx_local - 6, cy_local + 6, 4, 10),
                     border_radius=2)
    # Right leg
    pygame.draw.rect(body_surf, leg_col,
                     (cx_local + 2, cy_local + 6, 4, 10),
                     border_radius=2)

    # Head — skin-colored circle on top.
    pygame.draw.circle(body_surf, skin_col, (cx_local, cy_local - 14), 5)
    # Head shadow / outline.
    pygame.draw.circle(body_surf, outline_col, (cx_local, cy_local - 14), 5, 1)

    # Outline the whole silhouette.
    # (Cheap approach: trace the body rect + head + arms.)
    pygame.draw.rect(body_surf, outline_col,
                     (cx_local - 7, cy_local - 10, 14, 16),
                     1, border_radius=4)

    # Rotate. The body was drawn pointing up; we rotate by the death rotation
    # plus the player's facing angle at death time so the body falls in the
    # direction he was looking.
    facing = getattr(STATE.player, "angle", 0.0)
    total_rot = -math.degrees(facing) + STATE.death_player_rotation
    rotated = pygame.transform.rotate(body_surf, -total_rot)

    # Final screen position.
    sx = int(wx + slide_x - fxo)
    sy = int(wy + slide_y - fyo)
    rect = rotated.get_rect(center=(sx, sy))

    # Soft body shadow (drawn under the rotated body so it stays on the ground).
    shadow = pygame.Surface((rect.w + 6, rect.h + 4), pygame.SRCALPHA)
    pygame.draw.ellipse(shadow, (0, 0, 0, 70), (0, 0, rect.w + 6, rect.h + 4))
    surface.blit(shadow, (rect.x - 3, rect.y + 2))
    surface.blit(rotated, rect)
```